

Test Plan — Cryptonino (CTN)

1. Información General

Nombre del contrato: Cryptonino

Símbolo: CTN

Estándar: ERC-20

Red: Arbitrum

Estado: Desplegado en red real

Supply inicial: 9,000,000 CTN (18 decimales)

Mint: Único, en el constructor

Owner / Roles: No existen

Librería base: ERC20 de OpenZeppelin

2. Objetivo del Test Plan

Validar que el contrato **Cryptonino**:

- Cumple estrictamente el estándar ERC-20.
- Respetar las decisiones de diseño declaradas (supply fijo, sin owner, sin mint posterior).
- Es **trazable, verificable y auditable on-chain**.
- No expone funcionalidades implícitas o no documentadas.
- Presenta una **superficie de ataque mínima**.

3. Alcance

Incluido

- Deploy y verificación on-chain.
- Supply y eventos de mint.
- Transferencias ERC-20.
- Approve / transferFrom.
- Casos negativos.
- Análisis de seguridad aplicable.
- Evidencia y trazabilidad.

Excluido

- Frontend o UX.
- Seguridad de wallets.
- Phishing, social engineering o errores de usuario.
- Integraciones externas (DEX, bridges, etc.).

4. Estrategia de Pruebas

- **Tipo:** Funcional + Seguridad básica + Evidencia on-chain.
- **Nivel:** Smart contract.
- **Fuente de verdad:** Blockchain (Arbiscan, receipts, logs).
- **Criterio:** Verificación por lectura directa del estado y eventos.

5. Tests de Deploy

Objetivo:

Confirmar que el contrato fue desplegado correctamente y es operativo.

Casos cubiertos:

- El contrato tiene una dirección válida en Arbitrum.
- La transacción de deploy fue minada (`status = success`).
- Existe bytecode en la dirección del contrato.
- `totalSupply()` devuelve `9_000_000 * 1e18`.
- `balanceOf(deployer) == totalSupply`.

6. Tests de Supply

Objetivo:

Validar la **inmutabilidad del supply** y la correcta asignación inicial.

Casos cubiertos:

- El supply se define una sola vez en el constructor.
- El evento `Transfer(address(0), deployer, totalSupply)` existe.
- El supply no cambia tras transferencias.
- No existe ninguna función pública que permita mint adicional.
- El ABI no contiene `mint`, `burn`, `owner`, `pause`, ni roles.

7. Tests de Transferencias

Objetivo:

Validar el comportamiento estándar ERC-20.

Casos cubiertos:

- Transferencia válida entre dos cuentas.
- Actualización correcta de balances.
- Emisión del evento `Transfer`.
- Transferencia de valor 0 (edge case estándar).
- Transferencia mayor al balance (revert).
- Transferencia a `address(0)` (revert).

8. Tests de Approve / transferFrom

Objetivo:

Validar el sistema de allowances conforme a ERC-20.

Casos cubiertos:

- `approve` asigna correctamente allowance.
- Emisión del evento `Approval`.
- `transferFrom` respeta balances y allowance.
- Decremento correcto del allowance.
- `transferFrom` sin allowance (revert).
- Sobrescritura de allowance (comportamiento OpenZeppelin).
- Documentación del patrón seguro frente a race condition.

9. Tests Negativos

Objetivo:

Confirmar que el contrato **falla correctamente** ante usos inválidos.

Casos cubiertos:

- Transfer con balance insuficiente.
- `transferFrom` sin allowance.
- Transfer a `0x0`.
- Llamadas a funciones inexistentes (`mint`, `burn`, `pause`).
- Confirmación de ausencia total de owner o roles.

10. Tests de Seguridad

Objetivo:

Evaluar riesgos **aplicables** al contrato.

Evaluaciones:

- **Reentrancy:** No aplicable (sin llamadas externas).
- **Overflow / Underflow:** No aplicable (Solidity $\geq 0.8.x$).
- **Inflación maliciosa:** No aplicable (mint solo en constructor).
- **Privilege escalation:** No aplicable (no hay owner).
- **Front-running / phishing:** Fuera del alcance del contrato (documentado como riesgo de UX).

11. Verificación On-Chain

Objetivo:

Garantizar transparencia y auditabilidad.

Casos cubiertos:

- El contrato está verificado en Arbiscan.
- El código verificado coincide con el repositorio GitHub.
- La versión de Solidity y los imports coinciden.
- Los parámetros del constructor son correctos.

12. Evidencia Requerida

Debe existir en el repo:

- Dirección del contrato.
- Tx hash del deploy.
- Link de verificación en Arbiscan.
- Receipts relevantes (deploy, transfers).
- Logs de eventos ([Transfer](#), [Approval](#)).
- [docs/deployment.md](#) actualizado.
- [docs/design-decisions.md](#).
- Este [test-plan.md](#).

13. Criterio de Aceptación (QA Sign-off)

El contrato **Cryptonino (CTN)** se considera **APROBADO** si:

- El supply es fijo y verificable.
- El contrato no tiene funciones administrativas ocultas.
- Todas las operaciones ERC-20 funcionan según estándar.
- El código es completamente trazable on-chain.
- La documentación permite entender el proyecto sin contexto externo.

Notas:

Este proyecto demuestra:

- Criterio de diseño defensivo.
- Eliminación consciente de privilegios.
- Testing enfocado en riesgos reales.
- Comprensión del estándar ERC-20 y su superficie de ataque.
- Capacidad de documentar y auditar contratos en red real.